

Newsletter

This task provides you with experience in connecting a React frontend and NodeJS backend within your DEV@Deakin web platform.

Complete the following:

1. Complete the Task Requirements.
2. You will find “Topic Videos and Demo Videos” of Week 10 on the SIT313 CloudDeakin site particularly useful as a reference for this task.
3. Create a new [GitLab](#) repository for this task. Ensure all files/resources created are committed to this repository.
4. Create a 2 – 4 minute video where you provide a walkthrough of your code and solution, demonstrating your understanding of the concepts. Upload your video to [Panopto](#).
5. Create a short report that includes:
 - a. Screenshots of your DEV@Deakin web platform.
 - b. Screenshots of your terminal, showing a successful Status 200/202 message from your chosen Email API (hint: may need to use `console.log()`).
 - c. A link to your task repository on GitLab.
 - d. A link to your video.
6. **Upload** to OnTrack and **engage with feedback** to get this complete!

Task Requirements

1. GitLab repository

- Create a New Project called Task P5. Set the Visibility Level to **Private**.
- Once the project is created, under **Manage->Members** add the Unit Chair (Aaron Spence) and your OnTrack marking tutor **as a Reporter role**.

- Strictly do not submit your `node_modules` folder to your repository. To prevent this, create a `.gitignore` file, and add inside it `node_modules/` to prevent this folder from being tracked by Git.
- Commit all code and resources (e.g. images) that you develop for this task into this repository, with proper commit comments.

Failure to follow the above steps means your task cannot be marked as Complete.

2. Connect your Frontend and Backend

Continue with your DEV@Deakin web platform. Enable your React frontend to send and receive HTTP requests with the ExpressJS backend you developed in Task P2. You may re-use your original ExpressJS code from your Task P2 submission.

Using NextJS?

If you opted to build your web platform in NextJS, you may continue to use NextJS as both the frontend and backend (and do not need to use ExpressJS as your backend).

Tip: Organising Your Project

Option #1: It can be recommended to place your React frontend project and your ExpressJS backend project in *separate* folders, for them to exist and operate independently from each other. This may lead to duplication of files (e.g. `node_modules`, `.env`), but can be easier to manage.

Option #2: Bundle your frontend and backend together by having your ExpressJS serve your React frontend files instead. Has the benefit of having all code derived from the same origin, but may make large projects a bit more confusing to manage.

The native JavaScript `fetch()` library can be sufficient for all HTTP requests. However, you are encouraged to explore more feature-rich libraries such as:

- [TanStack Query](#)
- [SWR](#)
- [Axios](#)

You must setup correct handling of HTTP requests, including:

- Proper [HTTP response status codes](#)
- Proper validation of all data sent through HTTP requests (see the **Optional Technologies** section)
- Error handling with *try-catch* statements
- Messages/alerts to inform the user the outcome of each HTTP request (Did it fail? Did it succeed? Why?)

Cross-Origin Resource Sharing (CORS)

If your frontend and backend are separate and operate in isolation, then HTTP requests made from your frontend to your backend may be blocked by CORS. This occurs when a request to a service originates from a different location than itself – learn more [here](#). To resolve, tell ExpressJS to accept requests from either all origins with `app.use(cors())` or specific origins.

3. Implement the Sign-Up Feature

In Task P2 we implemented on the home page an email subscription for our users:



SIGN UP FOR OUR DAILY INSIDER

Upon clicking Subscribe, an Email API of your choice sent an email to the user. Back then we had a HTML/CSS frontend integrated well with ExpressJS.

Re-implement this feature with your current React-based DEV@Deakin web platform.

Optional Technologies

We strongly encourage you to explore additional technologies that are often standard in industry, that may assist you in completing this task:

- [Nodemon](#) (Node Monitor) package – Automatically restarts your ExpressJS server each time code changes are detected. Highly recommended.
- [TypeScript Execute \(tsx\)](#) – Acts the same as Nodemon, but is for ExpressJS servers built with TypeScript.
- [Zod](#) – An excellent library for data validation, such as usernames, emails and passwords. Robust data validation skills are crucial in industry.
- [TypeScript](#) – Adds ‘type safety’ to JavaScript. Standard JavaScript is not type-safe, meaning you can store any datatype in any variable which often leads to bugs. Strongly recommended to explore, either within this task or in your own time, and is expected to use within industry.

Know of some awesome technologies? Let us know in the SIT313 Teams channel!

Optional VS Code Extensions

VS Code has a gigantic library of extensions that can improve and speedup the development process. Here are some that may assist you:

- Tailwind CSS IntelliSense by Tailwind Labs – If using Tailwind CSS, this is a must-have extension to enable auto-complete when typing.
- REST Client by Huachao Mao – Send HTTP requests directly within VS Code; great for testing and debugging your backend.
- Prettier by prettier.io – Automatically formats your codes’ style and structure, to enforce a consistent style throughout.

Optional NPM Packages

There is a rich ecosystem of amazing packages to enhance our development. Here are some that may assist you:

- [SweetAlert2](#) – Beautiful, responsive, easy-to-use user popups.
- [React-Toastify](#) – Nice and easy user notifications.

Know of some awesome NPM packages? Let us know in the SIT313 Teams channel!

Caution: Beware of malicious NPM packages!

Unfortunately, NPM packages are now an active area infiltrated by malicious actors. NPM packages may appear 'safe' at glance, but may have embedded malicious code. Before installing any NPM package, investigate whether it is from a trustworthy author(s).

You are responsible for your own cyber-safety.

Available Resources

We have many resources to assist in your learning journey – if you are ever unsure then please reach out to us, we are always happy to help.

- **The SIT313 Teams channel** – an active learning resource with weekly announcements from the Unit Chair, and regular discussions/assistance from the SIT313 teaching team and students.
- **Seminars** – you can attend both on-campus and online sessions, and join as many as you like each week. Your tutors will provide demonstrations and assistance to help guide your learning.
- **SIT313 CloudDeakin** – contains a large collection of text and video resources.
- **Announcements** - Keep an eye on your email and any announcements made in the SIT313 Teams channel.

Caution: Acceptable use of Artificial Intelligence

It is acceptable to use AI as a learning tool. However, you can **never** ask AI to write code to complete some/parts of this task for you: this is cheating because it does not demonstrate that you understand anything. Detection of inappropriate use of AI has a very real consequences, such as failing the task, or even failing the entire unit.